K-Nearest Neighbors (KNN)

1. What Is K-Nearest Neighbors (KNN)?

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for both classification and regression. It predicts the output of a new data point by examining nearby points in the training dataset.

Unlike Linear Regression, which learns a global mathematical relationship, KNN relies on local proximity.

Classification

Classification predicts a category or class label. For example, KNN can classify a fruit as an apple or an orange by finding the K closest labeled fruits and selecting the majority class.

Regression

Regression predicts a continuous numerical value. For example, KNN can predict a house price by averaging the prices of the K most similar houses.

Task	Prediction method	Example
Classification	Majority voting	Predict whether a patient has diabetes
Regression	Average target value of neighbors	Predict a house price

2. How Does KNN Work?

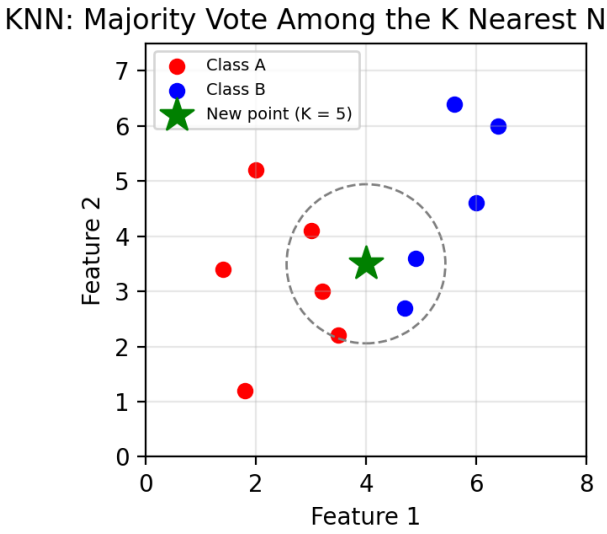
1. **Choose K:** Decide how many neighboring data points participate in the prediction.
2. **Calculate distances:** Measure the distance between the new point and each training point.
3. **Find the nearest neighbors:** Sort distances and select the K smallest.
4. **Predict:** Use majority voting for classification or the mean target value for regression.

Why is KNN called a lazy learner?

KNN is called a **lazy learner** because it generally does not learn model parameters during a conventional training phase. It stores the training examples and performs distance calculations when a prediction is requested. This is simple, but predictions can become expensive for large datasets.

3. Understanding the Value of K

K is the number of nearest neighbors used for a prediction. If five neighbors contain three Class A examples and two Class B examples, majority voting predicts Class A.



K value	General behavior
K = 1	Depends on the single closest point; sensitive to noise
Small K	More sensitive to local patterns and outliers
Large K	Uses a broader region and may smooth away local patterns
Very large K	May favor the majority class and underfit local structure

For binary classification, an odd K is often chosen to reduce the chance of a voting tie. However, an odd value does not guarantee that ties are impossible in multiclass classification.

There is no universally best K. Select it using validation data or cross-validation.

4. Mathematical Foundations: Distance Metrics

Let two points be:

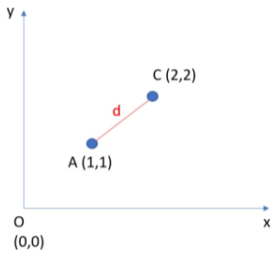
$$A = (x_1,x_2,\ldots,x_n),\ B = (y_1,y_2,\ldots,y_n)$$

Here, n is the number of features or dimensions.

4.1 Euclidean Distance

Euclidean distance is the straight-line distance between two points. In two dimensions, it follows from the Pythagorean theorem.

$$d_E(A,B) = \sqrt{(\Sigma(i = 1 \text{ to } n)\ (x_i - y_i)^2)}$$



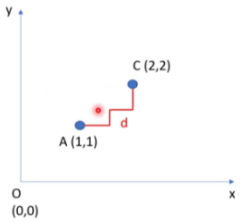
For two dimensions:

$$d_E(A,B) = \sqrt{((x_1 - y_1)^2 + (x_2 - y_2)^2)}$$

Example: Let `A = (2, 3)` and `B = (5, 7)`.

$$d_E(A,B) = \sqrt{((2-5)^2 + (3-7)^2)} = \sqrt{(9+16)} = \sqrt{25} = 5$$

4.2 Manhattan Distance



Manhattan distance, also known as taxicab distance, sums the absolute differences across coordinates.

$$d_M(A,B) = \Sigma(i = 1 \text{ to } n)\ |x_i - y_i|$$

For the same points:

$$d_M(A,B) = |2-5| + |3-7| = 3 + 4 = 7$$

Property	Euclidean	Manhattan
Measurement	Straight-line distance	Sum of absolute coordinate differences
Formula	`sqrt(sum((x_i-y_i)^2))`	`sum(abs(x_i-y_i))`
Effect of large differences	Squaring emphasizes them	Absolute differences are added linearly
Common interpretation	Geometric proximity	Grid-like or coordinate-wise movement

Manhattan distance can be **useful in** some **high-dimensional** settings, but it is not automatically better than Euclidean distance for every dataset. Validate the choice.

5. Calculate Distances Using Python and NumPy

5.1 Direct NumPy implementation

```
import numpy as np

A = np.array([2, 3])
B = np.array([5, 7])

euclidean = np.sqrt(np.sum((A - B) ** 2))
manhattan = np.sum(np.abs(A - B))

print("Euclidean distance:", euclidean)
print("Manhattan distance:", manhattan)
```

Expected output:

```
Euclidean distance: 5.0
Manhattan distance: 7
```

5.2 Generalize to any number of dimensions

These functions work for points with any number of features, provided both points have the same number of coordinates.

```
import numpy as np



def euclidean_distance(point1, point2):
    distance = 0

    for i in range(len(point1)):
        distance += (point1[i] - point2[i]) ** 2

    return np.sqrt(distance)



def manhattan_distance(point1, point2):
    distance = 0

    for i in range(len(point1)):
        distance += abs(point1[i] - point2[i])

    return distance



A = np.array([2, 3, 4])
B = np.array([5, 7, 1])

print("Euclidean:", euclidean_distance(A, B))
print("Manhattan:", manhattan_distance(A, B))
```

Expected output:

```
Euclidean: 5.916079783099616
Manhattan: 10
```

Understanding the loop:

- `range(len(point1))` visits every feature index.
- `point1[i] - point2[i]` calculates the difference for one feature.
- Euclidean distance squares each difference and takes the square root of their sum.
- Manhattan distance adds the absolute differences directly.

Smaller distance means points are closer under the chosen metric; larger distance means they are farther apart. Distance is a measure of proximity, not a guarantee that two samples have the same class.

6. Building a KNN Classifier from Scratch

This implementation uses NumPy, supports Euclidean and Manhattan distances, and predicts classes using majority voting.

6.1 Complete implementation

```
import numpy as np



class KNNClassifier:
    def __init__(self, k=3, distance_metric="euclidean"):
```

```
        if k < 1:
            raise ValueError("k must be at least 1")

        if distance_metric not in ("euclidean", "manhattan"):
            raise ValueError(
                "distance_metric must be 'euclidean' or 'manhattan'"
            )

        self.k = k
        self.distance_metric = distance_metric
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        self.y_train = np.asarray(y)

        if self.X_train.ndim != 2:
            raise ValueError("X must be a two-dimensional array")

        if len(self.X_train) != len(self.y_train):
            raise ValueError("X and y must have the same number of samples")

        if self.k > len(self.X_train):
            raise ValueError("k cannot exceed the number of training samples")

        return self

    def calculate_distance(self, point1, point2):
        if self.distance_metric == "euclidean":
            return np.sqrt(np.sum((point1 - point2) ** 2))

        return np.sum(np.abs(point1 - point2))

    def predict_one(self, test_point):
        test_point = np.asarray(test_point, dtype=float)
        distances = []

        for i, train_point in enumerate(self.X_train):
            distance = self.calculate_distance(test_point, train_point)
            distances.append((distance, self.y_train[i]))

        distances.sort(key=lambda item: item[0])

        nearest_labels = [
            label for _, label in distances[:self.k]
        ]

        # Ties are resolved by the first label in np.unique's sorted order.
        labels, counts = np.unique(nearest_labels, return_counts=True)
        return labels[np.argmax(counts)]

    def predict(self, X):
        X = np.asarray(X, dtype=float)

        if X.ndim == 1:
            return self.predict_one(X)
```

```
        return np.array([self.predict_one(point) for point in X])

    def score(self, X, y):
        predictions = self.predict(X)
        y = np.asarray(y)
        return np.mean(predictions == y)


# Example training data: two features and two classes
X_train = np.array([
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 5],
    [7, 7],
    [8, 6]
], dtype=float)

y_train = np.array(["A", "A", "A", "B", "B", "B"])

model = KNNClassifier(k=3, distance_metric="euclidean")
model.fit(X_train, y_train)

new_point = np.array([2, 2])
prediction = model.predict(new_point)

print("Predicted class:", prediction)
print("Training accuracy:", model.score(X_train, y_train))
```

6.2 Understanding the implementation

Component	Purpose
`__init__()`	Sets K and the distance metric
`fit()`	Stores training features and labels
`calculate_distance()`	Calculates Euclidean or Manhattan distance
`distances.append(...)`	Stores each distance with its label
`distances.sort(...)`	Orders points from nearest to farthest
`distances[:self.k]`	Selects the K nearest neighbors
`np.unique(..., return_counts=True)`	Counts how often each class occurs
`predict_one()`	Predicts a class for one point
`predict()`	Predicts for one or multiple points
`score()`	Calculates the fraction of correctly classified samples

Tie-breaking note: `np.unique` sorts labels before resolving a voting tie. This educational implementation selects the first label in that sorted order. Other KNN implementations may use different tie-breaking rules.

6.3 Why is there no conventional training loop?

The `fit()` method simply stores the data. KNN does not learn weights through Gradient Descent. Most of the computation happens when `predict_one()` calculates and sorts distances. This is the practical meaning of a lazy learner.

7. Applying KNN to a Diabetes Dataset

The fifth video summary describes applying KNN to a diabetes dataset and comparing accuracy for different K values. The summary does not provide the exact column names, split settings, or numerical accuracy results, so those details cannot be reproduced exactly.

The following general example assumes a CSV file named `diabetes.csv` and a target column named `Outcome`. Other columns are used as input features. It uses scikit-learn for splitting, scaling, and evaluation, while the custom KNN class above performs classification.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = pd.read_csv("diabetes.csv")

X = data.drop(columns=["Outcome"]).to_numpy(dtype=float)
y = data["Outcome"].to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Fit scaling on training data only to avoid test-data leakage.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

for k in [3, 5, 7, 9]:
    model = KNNClassifier(k=k, distance_metric="euclidean")
    model.fit(X_train, y_train)

    train_predictions = model.predict(X_train)
    test_predictions = model.predict(X_test)

    train_accuracy = accuracy_score(y_train, train_predictions)
    test_accuracy = accuracy_score(y_test, test_predictions)

    print(f"K = {k}")
    print(f"Training accuracy: {train_accuracy:.3f}")
    print(f"Test accuracy:     {test_accuracy:.3f}")
    print()
```

Why is feature scaling important?

Suppose a dataset contains:

- Age, ranging roughly from 20 to 80.
- Glucose level, ranging roughly from 50 to 200.
- A binary feature, taking values 0 or 1.

Distance calculations can be dominated by features with larger numerical scales. Standardization puts numerical features on comparable scales so that units alone do not disproportionately affect distance.

The scaler must be fitted only on training data. Fitting it on the full dataset leaks information from the test set into preprocessing.

Interpreting the results

- **Training accuracy:** How well the classifier predicts samples used during fitting.
- **Test accuracy:** How well it predicts held-out samples.
- **Changing K:** Reveals the trade-off between sensitivity to local noise and excessive smoothing.

For imbalanced datasets, accuracy alone may be misleading. Also consider precision, recall, F1-score, and the confusion matrix.

8. Advantages and Disadvantages of KNN

Advantages

1. **Simple to understand and implement:** The main process is distance calculation followed by neighbor selection.
2. **Supports classification and regression:** The general idea can predict labels or numerical values.
3. **Can model non-linear decision boundaries:** Predictions depend on nearby observations rather than one global linear equation.
4. **Flexible distance metrics:** Different metrics can be selected for different tasks.
5. **Useful on smaller datasets:** It can be effective when the feature space is manageable and meaningful neighbors exist.

Disadvantages

1. **Sensitive to outliers and noisy data:** Nearby unusual points can affect predictions, especially when K is small.
2. **High-dimensional data can be challenging:** Distances may become less informative as feature count increases.
3. **Prediction can be expensive:** A basic implementation calculates distances to many or all training samples for every prediction.
4. **Sensitive to feature scales:** Scaling is often needed when features use different units or ranges.
5. **Class imbalance can bias voting:** A majority class may dominate the neighborhood and lead to poor minority-class predictions.

KNN is not inherently ineffective on every imbalanced dataset, but imbalance can make ordinary majority voting unreliable. Class weighting, distance-weighted voting, resampling, or suitable evaluation metrics may help.

9. Quick Revision Sheet

Question	Short answer
What does KNN stand for?	K-Nearest Neighbors
What type of learning is it?	Supervised learning
What tasks can it perform?	Classification and regression
What does K represent?	Number of neighbors considered
How does classification work?	Majority voting
How does regression work?	Commonly, the mean target value of neighbors
What is Euclidean distance?	Straight-line distance
What is Manhattan distance?	Sum of absolute coordinate differences
Why is KNN called a lazy learner?	It stores training examples and defers most computation until prediction
Why scale features?	To make distance comparisons less dependent on feature units
Why test different K values?	To find a suitable balance between local sensitivity and smoothing
Why can high dimensionality be a problem?	Distances may become less informative and computation increases

10. Test Yourself

1. Why is KNN classified as a supervised learning algorithm?
2. What is the difference between KNN classification and KNN regression?
3. Calculate Euclidean and Manhattan distances between A = (1,2) and B = (4,6).
4. Why is KNN called a lazy learner?
5. What can happen when K is too small? What can happen when K is too large?
6. Why does feature scaling matter for KNN?
7. Why does a basic KNN implementation sort distances?
8. Why can accuracy be misleading on an imbalanced diabetes dataset?
9. Why must a scaler be fitted on the training set rather than the full dataset?
10. Does choosing an odd K guarantee that a multiclass vote cannot tie?